PROGETTAZIONE DI SOFTWARE SICURO

Lezione 12
Analisi Statica del codice

Elvinia Riccobene

Università degli Studi di Milano - SSRI

V&V: Validazione e Verifica

Validazione (convalida): controlla che il sw sia conforme ai requisiti

- accertamento che il software soddisfi i requisiti dell'utente (in termini di funzionalità richieste)
- *building the right system*

Verifica: controlla che l'implementazione corrisponda alla sua specifica

- *building the system right*
- richiede un modello dei requisiti

Validazione a livello di implementazione

Le tecniche di V&V d'implementazione si dividono in:

Dinamiche: analizzano informazioni sul sw eseguendolo

Statiche: collezionano informazioni sul sw senza eseguirlo

Tecniche di Analisi dinamica

Runtime checkers

- analizzano l'esecuzione del codice per vedere se ci sono violazioni di memoria, deadlocks, ... prima di passare le chiamate al SO
- esempi: LibSafe, PurifyPlus, Immunix tools

Profilers

- Valutano le prestazioni
- Evidenziano comportamenti anomali dell'applicazione (non necessariamente scorretti)

Penetration testing tools

- Controllano (*scan*) l'applicazione secondo schemi di vulnerabilità conosciuti

Testing

- esecuzione di scenari per la ricerca di difetti
  - random testing o con criteri

Tecniche di Analisi Statica

- Compilazione
- Revisioni tra pari
- Ispezioni
- Walkthrough
- Analisi statica automatica del codice
- SAST
- Prove di correttezza
- Abstract Interpretation

Analisi statica in compilazione

I **compilatori** effettuano una analisi statica del codice per verificare che un programma soddisfi particolari caratteristiche di correttezza statica, per poter generare il codice oggetto.

- Tipiche anomalie identificabili:

– *nomi di **identificatori** non dichiarati,*

– *incoerenza tra **tipi** di dati coinvolti in una istruzione,*

– *incoerenza tra **parametri** formali ed effettivi in chiamate a subroutine,*

– *codice non **raggiungibile** dal flusso di controllo*

Dipende dal linguaggio

Esempi su Java

```java
public class EsempioCompilazione {

    public static void main(String[] args) {
        int x; // dichiarata ma NON inizializzata

        System.out.println(x); // ❌ ERRORE DI COMPILAZIONE
    }
}
```

```
error: variable x might not have been initialized
        System.out.println(x);
                           ^
```

```java
public class EsempioCompilazione2 {

    public static void main(String[] args) {
        String s = "ciao";
        s.toInt(); // ❌ ERRORE DI COMPILAZIONE: il metodo non esiste
    }
}
```

```
error: cannot find symbol
        s.toInt();
          ^
```

```java
public class EsempioCompilazione3 {

    public static void main(String[] args) {
        String numero = 10; // ❌ ERRORE DI COMPILAZIONE
    }
}
```

```
error: incompatible types: int cannot be converted to String
        String numero = 10;
                        ^
```

Revisione tra pari

Tecnica per valutare in modo anonimo il codice per la *qualità*, la *leggibilità*, l'*usabilità*

- A ogni persona che partecipa a una revisione tra pari viene chiesto di consegnare due programmi: un programma "migliore" e uno di qualità inferiore.
- Questi programmi vengono poi distribuiti casualmente tra i partecipanti.
- Ogni partecipante valuta quattro programmi: due programmi "migliori" e due programmi di qualità inferiore.
- Dopo che tutti i risultati sono stati raccolti, ogni partecipante riceve le valutazioni (anonime) dei propri programmi, nonché le statistiche dell'intero test.

Revisione tra pari

E' una pratica standard in:

- aziende di software, soprattutto in progetti open-source
- team agili
- sviluppo tramite GitHub/GitLab

È considerata essenziale per:

- migliorare la qualità del codice
- ridurre i bug
- condividere conoscenza nel team
- mantenere uno stile coerente
- fare mentoring tra sviluppatori

Strumenti moderni che la integrano automaticamente:

- GitHub Pull Request; GitLab Merge Request

Ispezione o *code reading*

Ispezione, poco tecnologica, ma efficace

Ispezione di codice secondo il modello Fagan

- tra le tecniche di ispezione di maggior successo
  - più formale e ben strutturata di molte altre proposte
- tecnica completamente manuale per trovare e correggere errori
- è estensibile a tutti gli artefatti del ciclo di vista del SW: progetto, requisiti, ... seguendo principi organizzativi analoghi

Modello Fagan: ruoli nell'ispezione di sw

Moderatore:

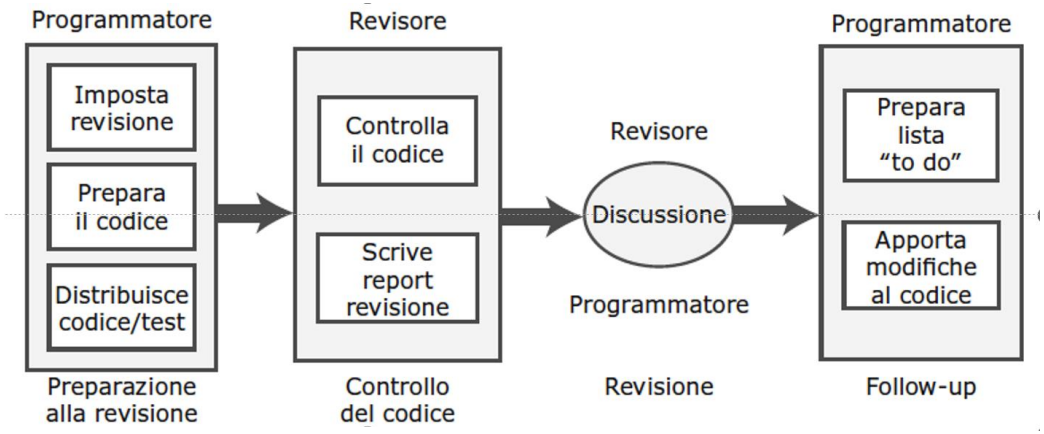
- responsabile dell'organizzazione delle riunioni di ispezione
- presiede le sedute, sceglie i partecipanti, controlla il processo

Revisori (2):

- leggono il codice
- cercano difetti

Programmatore:

- partecipante passivo; risponde a domande quando richiesto



Modello di Fagan: processo d'ispezione

Attività	Descrizione
Imposta revisione	Il programmatore contatta un revisore e concorda una data.
Prepara il codice	Il programmatore raccoglie il codice e i test per la revisione e annota entrambi con informazioni per il revisore sullo scopo inteso.
Distribuisce codice/test	Il programmatore invia codice e test al revisore.
Controlla il codice	Il revisore controlla sistematicamente il codice e i test sulla base della sua comprensione di ciò che dovrebbero fare.
Scrive un report della revisione	Il revisore annota il codice e i test con un report dei problemi da discutere all'incontro di revisione.
Discussione	Revisore e programmatore discutono i problemi e si accordano sulle azioni per risolverli.
Prepara lista "to do"	Il programmatore documenta l'esito della revisione sotto forma di lista "to do" e la condivide con il revisore.
Apporta modifiche al codice	Il programmatore modifica il codice e i test per risolvere i problemi rilevati dalla revisione.

Modello Fagan: Checklist

Checklist divise per:

- funzionalità, uso dei dati, flusso di controllo, connessioni, calcolo, manutenzione, chiarezza del codice

Esempi di valutazioni:

- uso improprio dei dati: variabili non inizializzate, indice di array fuori limite, puntatori dangling;
- errori nelle dichiarazioni: uso di variabili non dichiarate o dichiarazione omonima in blocchi annidati;
- errori di calcolo: divisione per zero, overflow (possibile anche in risultati intermedi), uso errato di variabili di diverso tipo nella stessa espressione, errori causati da un'errata comprensione delle priorità degli operatori;
- difetti nelle espressioni relazionali: utilizzo di un operatore errato (> invece di ≥, = invece di ==) o comprensione errata delle priorità degli operatori booleani;
- errori nel flusso di controllo: loop infiniti o un loop che viene eseguito n + 1 o n − 1 volte anziché n volte;
- errori nelle interfacce: numero errato di parametri, parametri di tipo errato o utilizzo incoerente delle variabili globali.

Modello Fagan: Checklist

Altri elementi *più progettuali* da valutare:

- il codice corrisponde alle specifiche richieste?
- gli usi non standard sono isolati in sottoprogrammi e ben documentati?
- i commenti sono sufficienti per capire il codice?

Per evitare la superficialità dei test di valutazione, i revisori devo rispondere a domande del tipo:

- "in quali condizioni può essere chiamata questa funzione? "
- "qual è l'effetto di questa funzione sul comportamento di altre funzioni?" (analisi delle dipendenze)

Esempio: Checklist per codice Java

Java Inspection Checklist

Disponibile su myariel

Ispezione - attenzione

Difetti trovati nell'ispezione non sono usati nella valutazione personale

- i programmatori non hanno motivo di nascondere i difetti

Difetti trovati durante il test (dopo l'ispezione) sono usati nella valutazione personale

- i programmatori sono incentivati a trovare difetti nell'ispezione

Walkthrough

Analisi informale del codice svolta da vari partecipanti i quali '*operano come il computer*':

- si scelgono alcuni casi di test e si simula l'esecuzione del codice a mano (si attraversa- *walkthrough-* il codice).

L'organizzazione della riunione è simile a quella della tecnica delle Ispezioni:

– Tra 3 e 5 partecipanti, ma non ruoli rigidi

– Riunioni brevi (max. 120 minuti);

– Attenzione sulla ricerca dei difetti, piuttosto che sulla correzione.

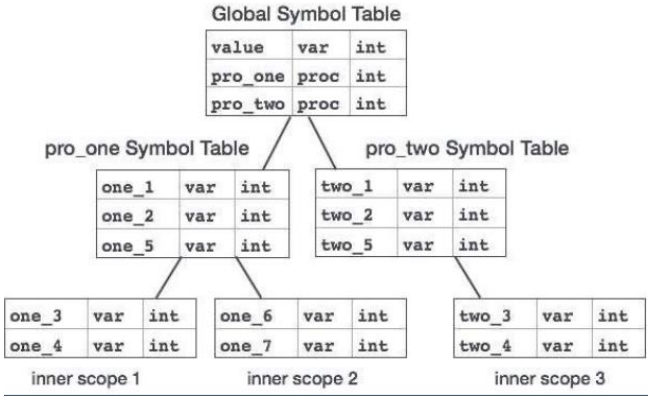
Differenze: informalità e l'autore che guida la revisione

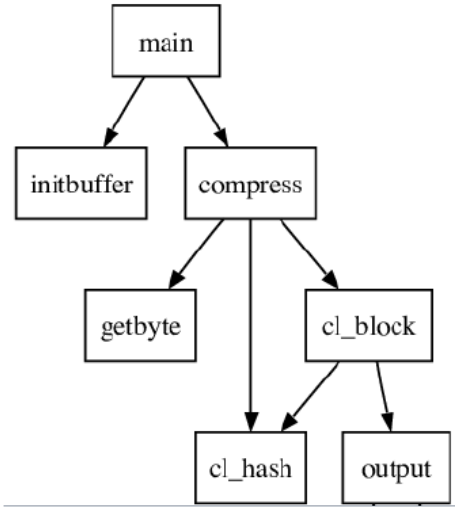
Analisi statica del codice

Analisi del flusso di controllo e del flusso dei dati

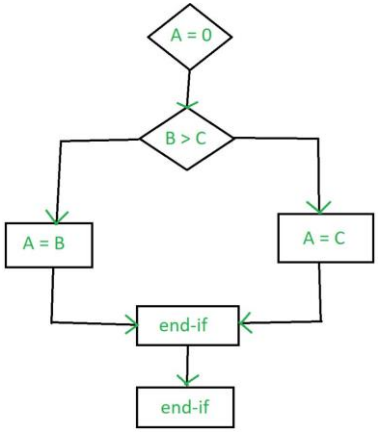
- Usa modelli di rappresentazione del sw in termini di grafi: Control graph, Symbol table, Call graph, Abstract syntax tree, ecc.

Symbol table

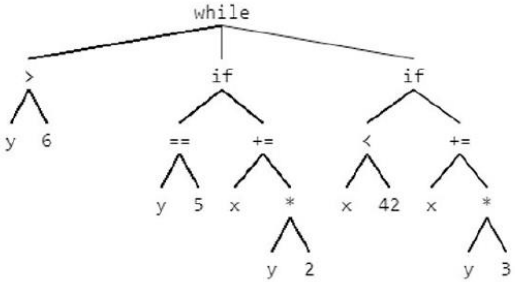




Call graph



Control graph



Abstract syntax tree

Analisi statica del codice

- Tecnica usata già dal compilatore per ottimizzare
- Valuta la qualità del codice (non solo correttezza)
  - Le metriche che sono prese in esame sono:
    1. Stile del codice (aderire a convenzioni)
    2. Costrutti sospetti (Code smell)
    3. Codice duplicato
    4. Accoppiamento
    5. Complessità

- Esistono strumenti dedicati:
  - PMD: analisi statica di Java, Python, C, C++, etc.
  - SpotBugs: Eclipse plugin di analisi statica di Java
  - FindBugs: analisi statica per la ricerca di vulnerabilità
  - SonarLint : fa parte di una suite molto completa (Sonarqube)
  - Snyk: tool per analisi statica basato su intelligenza artificiale

Esempio di codice Java

```java
public class EsempioAnalisiStatica {

    public static void main(String[] args) {
        String testo = null; // valore potenzialmente pericoloso

        if (args.length > 0) {
            testo = args[0];
        }

        // ⚠ Se non passo argomenti, `testo` rimane null
        System.out.println("Lunghezza del testo: " + testo.length());
    }
}
```

Se eseguito senza argomenti, a runtime si ha l'errore:

```
Exception in thread "main" java.lang.NullPointerException
    at EsempioAnalisiStatica.main(EsempioAnalisiStatica.java:11)
```

Esempio di codice Java

```java
public class EsempioAnalisiStatica {

    public static void main(String[] args) {
        String testo = null; // valore potenzialmente pericoloso

        if (args.length > 0) {
            testo = args[0];
        }

        // ⚠ Se non passo argomenti, `testo` rimane null
        System.out.println("Lunghezza del testo: " + testo.length());
    }
}
```

Un tool di analisi statica (tipo SpotBugs, SonarQube, PMD, ecc.) avverte:

“Possibile dereferenziazione di un valore null (nome) alla riga …”
Categoria tipica: **Possible null dereference** / **NP_NULL_DEREFERENCE**

Control Flow Analysis

Il flusso di controllo è esaminato per verificarne la correttezza.

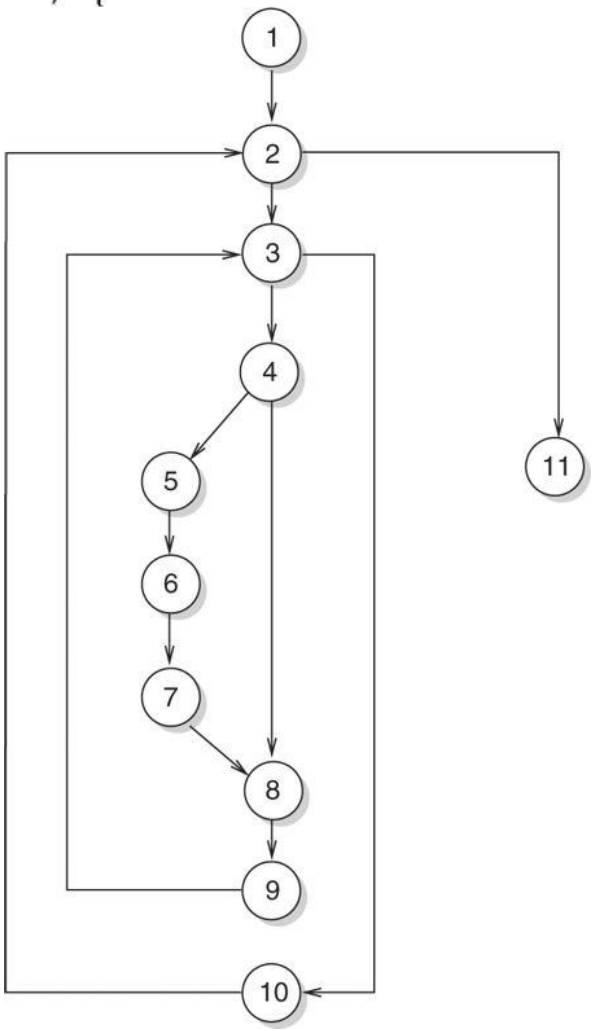
- Il codice è rappresentato tramite un grafo:

  Control flow Graph – **CfG** (grafo del flusso di controllo), i cui

  - nodi rappresentano statement (istruzioni e o predicati) del programma e gli
  - archi il passaggio del flusso di controllo.
- Un CfG è esaminato per :
  - identificare ramificazioni del flusso di controllo,
  - verificare l'esistenza di eventuali anomalie quali codice irraggiungibile e non strutturazione.

Esempio di CfG

```
1    public static void sort(int x []) {
2        for (int i = 0; i < x.length - 1; i++) {
3            for (int j = i+1; j < x.length;
4                if (x[i] > x[j]) {
5                    int save = x[i];
6                    x[i] = x[j];
7                    x[j] = save;
8                }
9            }
10       }
11   }
```



Vantaggi del CFG:

- Permette di vedere come viene eseguito il codice
- Aiuta a trovare gli errori:
  - rileva <u>codice irraggiungibile</u> o loop infiniti.
- Utile per l'ottimizzazione: migliora le prestazioni del programma.

Esempio di codice Java

```java
import java.io.*;

public class Main {
    public static void exampleFunction(int x) {
        if (x > 10) {
            System.out.println("x is greater than 10");
        } else {
            System.out.println("x is less than or equal to 10");
            return;
        }
    }

    public static void main(String[] args) {
        exampleFunction(15);
    }
}
```

CfG di `exampleFunction`

```
      B1: Start exampleFunction
        |
   if (x > 10)
    /       \
True        False
  |           |
 B2          B3
  |           |
 End         End
```

Legenda dei blocchi

Nodo e blocchi del CFG:

1. **B1**: Inizio della funzione `exampleFunction`, che include il controllo della condizione `if (x > 10)`.
2. **B2**: Ramo vero del `if`, che esegue `System.out.println("x is greater than 10")`.
3. **B3**: Ramo falso del `if`, che esegue `System.out.println("x is less than or equal to 10")` e termina con `return`.

Connessioni tra i nodi:

- **B1 → B2**: Transizione dal nodo di partenza al ramo vero (`if (x > 10)`).
- **B1 → B3**: Transizione dal nodo di partenza al ramo falso (`else`).
- **B2 → End**: Dopo aver stampato `"x is greater than 10"`, la funzione termina.
- **B3 → End**: Dopo aver stampato `"x is less than or equal to 10"` e il `return`, la funzione termina.

Codice modificato con errore

```java
public class Main {

    public static void exampleFunction(int x) {

        if (x > 10) {
            System.out.println("x is greater than 10");
            return;
        } else {
            System.out.println("x is less than or equal to 10");
            return;
        }

        // Istruzione irraggiungibile
        System.out.println("This statement is unreachable");
    }

    public static void main(String[] args) {
        exampleFunction(15);
    }
}
```

CfG del codice modificato

```
      B1: Start & if (x > 10)
       /        \
   True          False
    |              |
  B2:           B3: Print "x <= 10" & return
Print "x > 10" & return
    |              |
   End            End
     \
      (Unreachable)
       B4: Print "Unreachable"
```

Legenda dei blocchi

Nodo e blocchi del CFG:

1. **B1**: Inizio della funzione `exampleFunction`, che include il controllo della condizione `if (x > 10)`.
2. **B2**: Ramo vero del `if`, che esegue `System.out.println("x is greater than 10")`.
3. **B3**: Ramo falso del `if`, che esegue `System.out.println("x is less than or equal to 10")` e termina con `return`.
4. **B4**: Nodo irraggiungibile che esegue `System.out.println("This statement is unreachable")`.
5. **End**: Nodo finale che rappresenta l'uscita dalla funzione.

Connessioni tra i nodi:

- **B1 → B2**: Transizione dal nodo di partenza al ramo vero (`if (x > 10)`).
- **B1 → B3**: Transizione dal nodo di partenza al ramo falso (`else`).
- **B2 → End**: Dopo aver stampato `"x is greater than 10"`, la funzione termina.
- **B3 → End**: Dopo aver stampato `"x is less than or equal to 10"` e il `return`, la funzione termina.
- **B4**: Non ci sono archi che portano a questo nodo, rendendolo **irraggiungibile**.

Data flow analysis - statica

Analisi dell' evoluzione del valore delle variabili durante l'esecuzione di un programma, permettendo di rilevare anomalie.

- Intrinsecamente è dinamica, ma alcuni aspetti possono essere analizzati staticamente.

Data flow analysis - statica

L'analisi statica data flow è legata alle operazioni eseguite su una variabile:

- definizione **(d)**: alla variabile è assegnato un valore
- uso **(u)**: il valore della variabile è usato in un'espressione o in un predicato
  - Es.: nell'espressione x:=y+z; la variabile x è *definita* mentre y e z sono *usate*
- annullamento **(a)**: al termine di un'istruzione il valore associato alla variabile non è più significativo

- La definizione **(d)** di una variabile **cancella** l'effetto di una precedente definizione della stessa variabile, perché ad essa è associato il nuovo valore derivante dalla nuova definizione
- Un annullamento **(a)** di una variabile **cancella** l'effetto di una precedente definizione perché ad essa è associato nuovo valore nullo

Data Flow Analysis

Una corretta sequenza di operazioni prevede che:

- L'uso **(u)** di una variabile sia sempre preceduto da una definizione della stessa variabile senza annullamenti intermedi (Sequenza valida : du)
  - *Una variabile non definita ha un valore "sporco"*
- Una definizione **(d)** di una variabile sia sempre seguita da un uso della variabile, prima di un'altra definizione o di un annullamento della stessa variabile (Sequenze non valide: dd, da)
  - *Una doppia definizione è indice del fatto che la prima definizione è risultata inutile*

Data Flow Analysis

Sequenze di istruzioni sono riconducibili a sequenze d,u,a:

```
Procedure swap (x1, x2: real)
var x:real
begin
   x2:=x;
   x2:=x1;
   x1:=x;
end;
```

x: (auu)

x2: (ddd)

x1: (dud)

Le sequenze

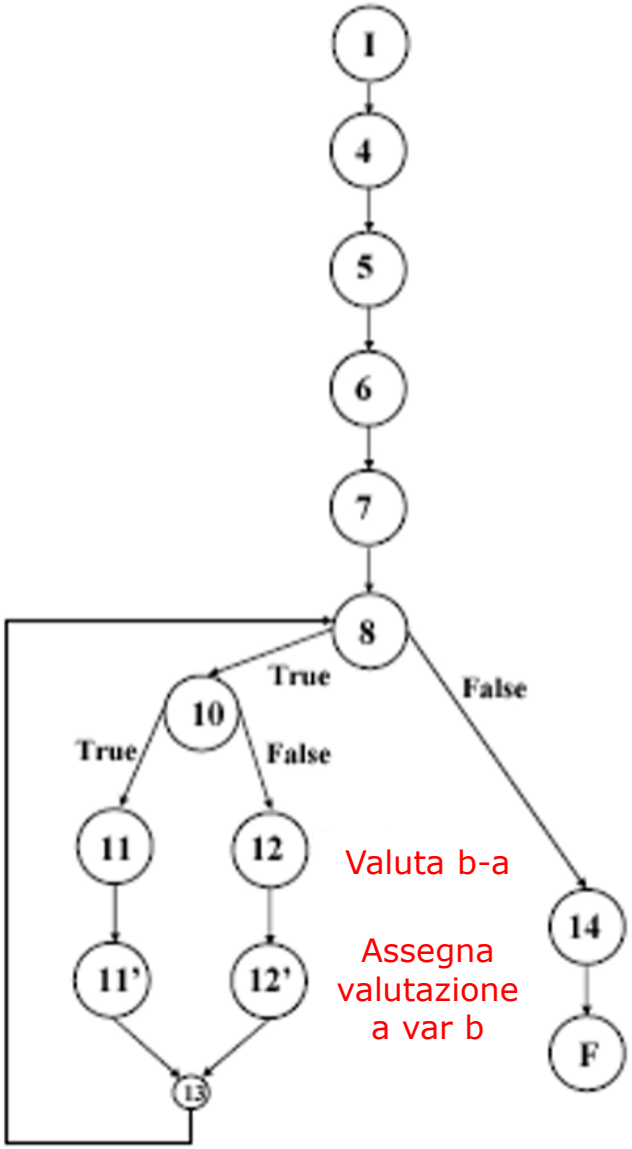
- auu su x e
- ddd su x2

sono indicative di qualche anomalia

La sequenza dud per x1 è accettata

Data Flow Analysis

```
1 program mcd (input, output);
2 var
3 x,y,a,b : integer;
4 begin
5 read (x,y);
6 a:=x;
7 a:=y
8 while a <> b do
9     begin
10        if a>b
11                then a:=a-b
12                else b:=b-a;
13   end;
14 write ("il massimo comune divisore è", a)
15 end.
```



Data Flow Analysis

Ad ogni nodo del CfG è possible associare l'insieme delle variabili in esso *definite*, *usate* ed *annullate*

Nodo	Var. definite	Var usate	Var. annullate
4			x, y, a, b
5	x, y		
6	a	x	
7	a	y	
8		a, b	
10		a, b	
11		a, b	
11'	a		
12		a, b	
12'	b		
14		a	

Per ogni variabile è possible definire la sequenza [d,u,a] facendo riferimento a tali insiemi e valutare sequenza anomale

Anomalia -dd per a
Anomalia a---u per b dovuta ad un errore a linea 7

P([I,4,5,6,7,8,10,11,11',8,14,F];*a*) = (-a-dduuuduu-)

P([I,4,5,6,7,8,10,11,11',8,14,F];*b*) = (-a---uuu-u--)

Analisi statica del codice (con CfG e DfA)

- Esempi di controlli:
  - istruzioni non raggiungibili
  - variabili usate prima di essere inizializzate
  - variabili dichiarate e mai usate
  - variabili scritte e mai lette
  - variabile mai usata tra due assegnamenti
  - possibili violazioni di array

- Le vulnerabilità che possono essere trovate sono:
  1. SQL Injection
  2. Cross-Site Scripting
  3. Buffer Overflow
  4. Remote Code Execution

SAST
Static Application Security Testing

Analisi statica del codice sorgente per trovare le vulnerabilità che espongono ad attacchi

- Gli **strumenti SAST** forniscono agli sviluppatori
  - un feedback in tempo reale durante la codifica, suggerendo la risoluzione dei problemi prima della compilazione
  - rappresentazioni grafiche dell'analisi, segnalando la posizione delle vulnerabilità
  - una guida approfondita su come risolvere i problemi

Limite: i software di scansione devono

- conoscere le peculiarità e le vulnerabilità dei linguaggi di programmazione usati
- avere meccanismi di aggiornamento continuo delle regole di rilevazione dei modelli non compliant

Tools per SAST

Checkmarx SAST e **Veracode**

Best for: Ideale per le aziende di medie e grandi dimensioni che cercano un solido supporto multilingue (Java, JavaScript e Python) e una perfetta integrazione CI/CD per ottimizzare le capacità di gestione delle minacce.

Spectral

Best for: Team DevOps, sviluppatori nativi del cloud e aziende attente alla sicurezza che necessitano di valutazioni della sicurezza in tempo reale nei flussi di lavoro di sviluppo.

Snyk Code

Best for: Se si sviluppa in open source, questo strumento offre un feedback tempestivo e si integra perfettamente con le pipeline di sviluppo.

Prova di Correttezza del Software

- È la tecnica di analisi statica più completa
- Obbiettivo: dimostrare che l'implementazione soddisfa la sua specifica
  - La specifica viene espressa formalmente in termini di due asserzioni che valgono rispettivamente *prima* e *dopo* l'esecuzione del programma
  - Si dimostra che: il programma S trasforma un'asserzione (la *precondizione* P) nell'altra (la *postcondizione* Q), ossia vale:

$$\{P\}\ S\ \{Q\}$$

(“Hoare triple”)

Esempio di correttezza

S: singola istruzione n := n + 9

P: n > 5

Q: n > 13

Correttezza: {n > 5} n := n + 9 {n > 14}

Contratto di SQRT

Sia SQRT una routine per il calcolo della radice quadrata

La correttezza di questa routine richiede la scrittura del contratto:

- S: ... algoritmo per calcolare y come radice quadrata di x (con precisione epsilon)
- P: l'input è positivo

$$\{x >= 0\}$$

- Q: y approssima esattamente il quadrato di x con un errore pari a epsilon

$$\{abs\ (y\ ^\wedge\ 2 - x) <= epsilon\}$$

Limiti della prova di correttezza

- Le prove di correttezza sono molto formali e sono difficili da costruire per il programmatore medio.
- La dimostrazione di correttezza è complessa e difficilmente effettuabile con strumenti automatici
- … a meno che il contratto non sia immerso nel codice:
  - Caso dei *Contratti JML* (o in generale del DbC)

Uso del contratto JML

1. **Documentazione**

   Un contratto può essere esportato in formato javadoc come parte della documentazione

2. **Controllo run-time**

   le condizioni possono essere attivate o disattivate a run-time (rallentano l'esecuzione)

   - attivate durante lo sviluppo e durante il testing
   - disattivate sul codice distribuito

3. **Prova di correttezza del software**

   dimostrare che il contratto è rispettato dall'implementazione

Correttezza di una Classe

Nel DBC una classe è corretta se sono corretti:

- operazione di creazione

per ogni nuova istanza INV vale

- $\{PRE_{costr}\}$ constructor $\{INV \wedge POST_{costr}\}$

- ogni altro metodo OP

se chiamo un metodo OP con PRE vero (e INV) allora dopo l'esecuzione del metodo vale POST e continua a valere INV

- $\{PRE_{op} \wedge INV\}$ OP $\{POST_{op} \wedge INV\}$

Abstract Interpretation

Tecnica di analisi statica basata sul concetto di astrazione:

- Partendo dalle istruzioni del codice, viene determinato il *dominio astratto del valore delle variabili*, definendo cosi' il (valore dello) stato astratto ad ogni istruzione (fino allo stato finale)

- L'analisi dello stato astratto permette di rilevare:

assenza di NullPointerException	limiti di array rispettati
assenza di divisioni per zero	variabili sempre inizializzate
assenza di dead code	sicurezza dei puntatori

Esempio semplice

x qualsiasi

```
int x = input();
int y;

if (x > 0) {
    y = 1;
} else {
    y = -1;
}

z = 10 / y;
```

Può essere y =0 ?

Simboli per domini astratti:
NEG = negativo
ZERO = zero
POS = positivo
TOP = “non so, può essere qualsiasi segno”

Stato astatto iniziale= x:TOP, y: ⊥ (sconosciuto/ non definito)

Stato astratto ramo then= x:POS, y:POS

Stato astratto ramo else= x : ZERO ∪ NEG, y:NEG

Join degli stati astatti: x:TOP, y: POS ∪ NEG

Stato finale: x:TOP, y: POS ∪ NEG, z: POS ∪ NEG

Staticamente possiamo concludere che:

divisione 10 / y è sicura, non ci sarà divisione per zero

LiSA

LiSA (Library for Static Analysis):

https://lisa-analyzer.github.io/

Una libreria open-source in Java che fornisce una infrastruttura completa per sviluppare analizzatori

Disponibile come strumento di analisi statica di vulnerabilità di tipo IoT per codice C++ e Java

Basato su abstract interpretation ed uso di CfG per sintetizzare la semantica di blocchi di codice